

Politecnico di Milano

Anno accademico 2013-2014

Ingegneria del Software – Appello del 28 giugno 2013

Cognome:		
Nome:		Matricola:
Sezione (segnarne una): ☐ Baresi	☐ Ghezzi	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si considerino le seguenti dichiarazioni:

```
public abstract class Point {
   public abstract double distance (Point p);
}

public class Point1D extends Point {
   private double c1;

   public Point1D (double c1) {
      this.c1 = c1;
   }

   public double getC1() {
      return c1;
   }

   public double distance (Point p) {
      return Math.abs(((Point1D) p).getC1()-c1);
   }
}

public class Point2D extends Point1D {
   private double c2;

   public Point2D (double c1, double c2) {
      super(c1);
      this.c2 = c2;
   }

   public double getC2() {
      return c2;
   }

   public double distance (Point p) {
      return Math.sqrt(Math.pow(((Point2D) p).getC1()-this.getC1(), 2) +
                       Math.pow(((Point2D) p).getC2()-this.getC2(), 2));
   }

   public double distance (Point2D p) {
      return Math.sqrt(Math.pow(p.getC1()-this.getC1(), 2) +
                       Math.pow(p.getC2()-this.getC2(), 2));
   }
}
```

Domanda 1

Quali dei due metodi distance della classe Point2D rappresenta un caso di overloading e quale invece rappresenta un caso di overriding? **Soluzione:** Il primo e' overriding, il secondo overloading,

Domanda 2

La classe Point2D è un sottotipo della classe Point1D in base al principio di sostituibilità? Motivare brevemente la risposta. **Soluzione:** I metodi non sono specificati in JML; tuttavia, essendo presente una implementazione, si può considerare il comportamento previsto dalla implementazione per ragionare sul principio di sostituibilità.

La classe Point2D non rispetta il principio di sostituibilità in quanto il metodo overridden distance ha un comportamento normale (non eccezionale) solo se il parametro p è instanceof Point2D. Se p è instanceof Point1D, ma non Point2D, il metodo

lancia una ClassCastException. Questo comportamento non è compatibile con quello di distance in Point1D, che invece ha un comportamento non eccezionale in entrambi i casi. Viene pertanto violata la regola dei metodi.

Domanda 3

Dopo la seguente sequenza di dichiarazioni:

```
Point p;
Point1D p1;
Point2D p2;
```

segnare a fianco di ciascuno dei seguenti assegnamenti se essi sono considerati corretti dal compilatore Java in base alle regole di tipo:

```
p = p1;

p = p2;

p1 = p;

p1 = p2;

p2 = p1;
```

Domanda 4

Si consideri il seguente frammento e si pieghi sinteticamente a parole l'effetto di ciascun assegnamento, indicando quali metodi vengono invocati in ciascun caso e scrivendo il risultato calcolato (valore stampato a video).

```
Point p1 = new Point1D(0.0);
Point p2 = new Point1D(1.0);
Point p3 = new Point2D(0.0, 1.0);
Point p4 = new Point2D(1.0, 0.0);
double x;

x = p1.distance(p2);
System.out.println(x);

x = p3.distance(p4);
System.out.println(x);

x = p1.distance(p3);
System.out.println(x);

x = p3.distance(p1);
System.out.println(x);

p1=p3;
x = p2.distance(p1);
System.out.println(x);

x = p4.distance(p1);
System.out.println(x);
```

Soluzione:

```
Point1D.distance
1.0
Point2D.distance
1.4142135623730951
Point1D.distance
0.0
Point2D.distance
```

```
Exception in thread "main" java.lang.ClassCastException: Point1D cannot be cast to Point2D

Se poi l'esecuzione proseguisse:
Point1D.distance
1.0
Point2D.distance
1.4142135623730951
```

Domanda 5

Si supponga di introdurre una classe Point3D sottoclasse di Point2D che definisce i punti in uno spazio a 3 dimensioni. Si esamini questo frammento di codice:

```
Point p1 = new Point3D(1.0, 1.0, 1.0);
Point2D p2 = new Point2D(0.0, 0.0);

p2.distance(p1);
```

spiegare sinteticamente che cosa succede durante l'esecuzione in corrispondenza della chiamata di *distance* e quale valore viene calcolato. **Soluzione:** chiama Point2D.distance e calcola la distanza fra $(1.0, 1.0)$ e $(0.0, 0.0)$.

Domanda 6

Si supponga di aggiungere la seguente precondizione al metodo distance della classe Point1D:

```
//@ requires p instanceof Point1D;
```

La classe Point2D sarebbe considerabile sottotipo di Point1D in base al principio di sostituibilità? Motivare sinteticamente la risposta. **Soluzione:** Ammettendo che il principio di sostituzione valesse senza la precondizione, la precondizione non violerebbe la regola dei metodi.

Domanda 7

Si supponga che, oltre alla precedente modifica della domanda 6, si aggiunga anche la seguente precondizione al metodo **distance (Point p)** della classe Point2D:

```
//@ requires p instanceof Point2D;
```

La classe Point2D sarebbe considerabile sottotipo di Point1D in base al principio di sostituibilità? Motivare sinteticamente la risposta. **Soluzione:** Ammettendo che il principio di sostituzione valesse senza la precondizione, la precondizione violerebbe la regola dei metodi, in quanto sarebbe piu' stringente della precondizione originale di Pont1D.

Domanda 8

E se, per assurdo, nel metodo distance di Point1D si introducesse la precondizione:

```
//@ requires p instanceof Point2D;
```

e nel metodo **distance (Point p)** della classe Point2D si introducesse:

```
//@ requires p instanceof Point1D;
```

la classe Point2D sarebbe considerabile sottotipo di Point1D in base al principio di sostituibilità? Motivare sinteticamente la risposta. **Soluzione:** Ammettendo che il principio di sostituzione valesse senza le precondizioni, questo caso non violerebbe la regola dei metodi.

Domanda 9

Si considerino le seguenti dichiarazioni che fanno riferimento allle dichiarazioni di classe fornite inizialmente (e cioè senza precondizioni):

```
ArrayList<Point1D> l1;
ArrayList<Point2D> l2;
```

Quali dei seguenti assegnamenti corretto secondo il compilatore Java, e perché?

```
l1 = l2;
```

```
l2 = l1;
```

Soluzione: Entrambi i casi sono considerati scorretti dal compilatore Java.

Esercizio 2

Si supponga di voler specificare una classe `Line`, che offre un metodo osservatore `points` che restituisce un `ArrayList<Point2D>`. Per semplicità, si ipotizzi che la classe rappresenti solo rette oblique.

1. Si specifichi un metodo `lineUp` che restituisce una nuova Line i cui punti sono ordinati per ascisse crescenti. La precondizione afferma che i punti dell'ArrayList appartengono a una retta; la postcondizione afferma che nell'oggetto restituito i punti memorizzati in posizioni successive dell'ArrayList si trovano in posizioni successive della retta. **Soluzione:** La signature del metodo non e' specificata. Diverse soluzioni sono possibili; per esempio, un producer che parta da un oggetto Line, oppure un creator che parta da un ArrayList di punti. Il fatto che i punti siano in linea puo' essere specificato dicendo che esiste un coeff. angolare m e un termine noto q per cui tutti i punti p sono disposti lungo la retta individuata da m e q:

   ```
   p.getC2() == m*p.getC1() + q
   ```

   .

   La specifica per un producer diventa:

   ```
   /*@ requires points != null && points.size() >= 2 &&
     @ (\exists double m; !Double.isNan(m) && !Double.isInfinite(m);
     @  (\exists double q; !Double.isNan(q) && !Double.isInfinite(q);
     @   (\forall Point2D p; p != null && points.contains(p);
     @     p.getC2() == m*p.getC1() + q)));
     @ ensures \result != null &&
     @ \result.containsAll(points) && points.containsAll(\result) &&
     @ (\forall int i; 0 <= i && i< \result.size() - 1;
     @  \result.points().get(i).getC1() < \result.points().get(i+1).getC1());
     @*/
   public static /*@ pure @*/ Line lineUp(ArrayList<Point2D> points);
   ```

   dove isNan, isInfinite sono utilizzati per evitare valori speciali dei Double. Si usa il "¡" per confrontare i valori dell'ascissa in quanto le rette sono per ipotesi (che si puo' immaginare garantita dal costruttore) oblique.

   Se si definisse il metodo con la signature di un creator:

   ```
   public /*@ pure @*/ Line lineUp();
   ```

   basta riscrivere la specifica precedente con this.points() al posto di points.

   In questo caso, in realtà, la proprietà di essere allineati dovrebbe essere garantita dall'invariante pubblico di Line.

2. Si fornisca la specifica del metodo al punto precedente eliminando la precondizione e sollevando invece al suo posto un'eccezione NoLineException. **Soluzione:**

   ```
   /*@ requires true;
     @ ensures points != null && points.size() >= 2 &&
     @ (\exists double m; !Double.isNan(m) && !Double.isInfinite(m);
     @  (\exists double q; !Double.isNan(q) && !Double.isInfinite(q);
     @   (\forall Point2D p; p != null && points.contains(p);
     @     p.getC2() == m*p.getC1() + q)));
     @ && \result != null && //qui riprende la ensures precedente
     @ \result.containsAll(points) && points.containsAll(\result) &&
     @ (\forall int i; 0 <= i && i< \result.size() - 1;
     @  \result.points().get(i).getC1() < \result.points().get(i+1).getC1());
     @ signals (NoLineException e) points == null || points.size() < 2 ||
     @ !(\exists double m; !Double.isNan(m) && !Double.isInfinite(m);
     @  (\exists double q; !Double.isNan(q) && !Double.isInfinite(q);
     @   (\forall Point2D p; this.points().contains(p);
     @     p.getC2() == m*p.getC1() + q)))*/
   public static Line lineUp(ArrayList<Point2D> points) throws NoLineException;
   ```

3. Con riferimento al punto precedente, si consideri una sottoclasse di `Line` in cui si voglia ridefinire il metodo `lineUp` nel modo seguente:

   - la precondizione, che corrisponde sia alla precondizione che alla postcondizione del caso precedente, richiede che i punti appartengano a una retta e i punti in posizioni successive dell'ArrayList siano consecutivi sulla retta,
   - la postcondizione specifica che l'oggetto contenga in successione solo i punti consecutivi della retta che hanno coordinate positive.

   Si chiede di definire la precondizione e la postcondizione e di discutere sinteticamente se la ridefinizione soddisfa il principio di sostituibilità.

Soluzione: Un modo per esprimere la condizione è il seguente. Dato un punto positivo nella lista dei punti,

- se il suo successivo è ancora positivo allora i due punti sono ordinati;
- altrimenti se il successivo non è positivo allora anche tutti i seguenti non sono positivi.

```
/*@ requires points.size() >= 2 &&
  @ (\exists double m; !Double.isNan(m) && !Double.isInfinite(m);
  @  (\exists double q; !Double.isNan(q) && !Double.isInfinite(q);
  @   (\forall Point2D p; points.contains(p);
  @     p.getC2() == m*p.getC1() + q)))
  @ &&
  @ (\forall int i; 0 <= i < points.size() - 1;
  @  points.get(i).getC1() <= points.get(i+1).getC1());
  @ ensures \result != null &&
  @ \result.containsAll(points) && points.containsAll(\result) &&
  @ (\forall int i; 0 <= i < \result.size() - 1;
  @ (\result.points().get(i)>0) => (
  @  ((\result.points().get(i+1)>0) &&
  @  \result.points().get(i).getC1() < \result.points().get(i+1).getC1())
  @  ||
  @ ((\result.points().get(i+1)<=0) &&
  @ \forall int k; i+1 < k < \result.points().size();
  @   (\result.points().get(k)<=0))));
  @*/
public Line lineUp(ArrayList<Point2D> points);
```

La nuova precondizione, detta pre_{sub} è più forte di quella del metodo originale, detta pre_{super}, quindi viene violato il principio di sostituzione.

La nuova postcondizione $post_{sub}$ è invece piu' debole di quella originale $post_{super}$, che imponeva a tutti i punti anche con coordinate non entrambe positive di essere ordinate nella linea. La violazione del principio di sostituzione e' comunque ulteriormente evidente, in quanto viene violata la regola completa della postcondizione: non e' vero che $pre_{super} \Rightarrow (post_{sub} \Rightarrow post_{super}$, in quanto non solo non vale $post_{sub} \Rightarrow post_{super}$, ma $post_{sub}$ puo' non essere nemmeno definita quando vale solo pre_{super}.

Esercizio 3

Si consideri il seguente frammento di programma C:

```
for (n=0; n<max_size && (c=getc(yyin)) != EOF && c!= '\n'; ++n)
buf[n] = (char) c;
```

1. Supponendo di voler coprire tutti i **branch**, qual'è il numero minimo di test che devono essere eseguiti? Giustificare la risposta. **Soluzione:** Il codice presenta un solo branch in corrispondenza del ciclo for per determinare se si deve uscire dal ciclo o se deve essere eseguita un'altra iterazione.

   Basta quindi un solo caso di test per cui le condizioni di ingresso nel ciclo sono verificate almeno una volta. Questo caso di test copre sia l'ingresso sia l'uscita dal ciclo che avverrà al più dopo `max_size` iterazioni.

2. Supponendo di voler anche coprire tutte le **condizioni**, quanti e quali test dovrebbero essere eseguiti? Specificare esattamente i casi di test, con i valori delle variabili

Soluzione: Chiamando `C1` la condizione `n < max_size`, `C2` la condizione `c != EOF` e `C3` la condizione `c != '\n'` , per poter coprire tutte le condizioni servono i seguenti casi di test.

- `C1 = False`
- `C1 = True && C2 = False`

- `C1 = True && C2 = True && C3 = False`
- `C1 = True && C2 = True && C3 = True`

L'operatore && in C, come in Java, è short circuited, quindi le altre possibili combinazioni delle condizioni non sono necessarie.

`getc` una funzione C che permette di leggere un carattere da un file, quindi i casi di test devono indicare il valore della variabile `max_size` e il contenuto del file cui fa riferimento `yyin`.

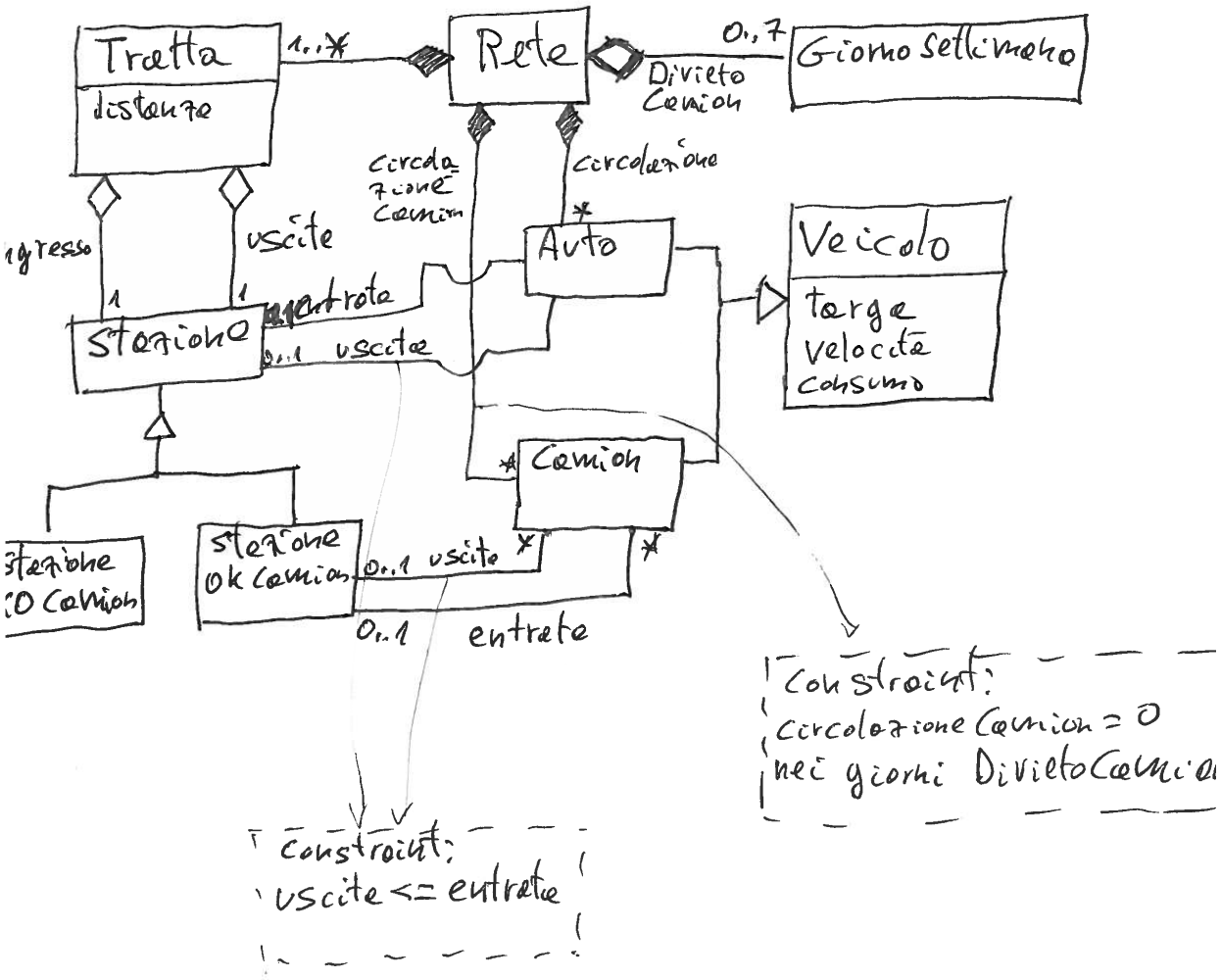
Dei possibili valori delle variabili per i diversi casi sono rispettivamente:

- `max_size = 0`
- `max_size = 1, yyin = riferimento a un file vuoto`
- `max_size = 1, yyin = riferimento a un file non vuoto, il cui prossimo carattere disponibile per la lettura è '\n'`
- `max_size = 1, yyin = riferimento a un file non vuoto, il cui prossimo carattere disponibile per la lettura è 'a'`

Esercizio 4

Si vuole progettare un programma di simulazione di una semplice rete autostradale. La rete è composta da stazioni di ingresso/uscita e tratte, che collegano due stazioni e che hanno una certa distanza. Un veicolo, da un punto di vista della simulazione, è caratterizzato da un tragitto, inteso come coppia stazione di ingresso, stazione di uscita, da una velocità e da un consumo medio di carburante per km. percorso. I veicoli posso essere automobili o camion. Questi ultimi possono avere dei divieti di circolazione in certi giorni della settimana. Alcune stazioni di ingresso/uscita possono essere vietate ai camion.

1. Specificare un diagramma delle classi che descrive questo sistema.
2. Si descriva un diagramma di sequenza che descrive il fatto che un veicolo per entrare in autostrada da una stazione deve ottenere il biglietto di ingresso e che arrivato alla stazione deve sottomettere prima il biglietto di ingresso e poi il pagamento relativo per poter uscire.
3. Si supponga di voler estendere il modello di simulazione per tener conto anche dei rifornimenti di benzina. Si usi uno Statechart per modellare il seguente comportamento. Una pompa di benzina funziona come segue: bisogna inserire la carta di credito e poi scegliere se si vuole prepagare l'importo (per esempio 30 EURO) o se si paga a consuntivo fino al pieno. In entrambi i casi, bisogna staccare la pompa e al termine dell'erogazione riattacarla. Nel caso di importo pagato a consuntivo, l'azione di riempimento del serbatoio è segnalata da un evento esterno (la pompa che rileva il pieno). Nel caso di prepagato invece l'azione ha una durata prefissata che non deve essere modellata.



Soluzione: Mostriamo solo il diagramma UML, le altre risposte essendo ovvie.